AI Service Architecture
Model Operating Overview

Company-facing summary for business and management review

Recommended model stack	Ministral 3 8B for orchestration and business reasoning, plus Devstral Small 2 24B for engineering and test-fix execution.

1. Executive summary

The recommended AI service uses a two-model local architecture. Ministral 3 8B acts as the orchestration and reasoning layer, while Devstral Small 2 24B acts as the engineering execution layer. This keeps the service simple enough for company operations while still supporting business analysis, controlled code changes, and structured testing workflows.

- **Main idea:** use one smaller decision model and one stronger software-engineering model, instead of many separate model deployments.
- **Operational goal:** keep all AI execution local, behind the company backend, with clear audit, validation, and approval points.
- **Profile model:** run about 30 local agent profiles on top of 2 base models, rather than deploying 30 separate models.

2. High-level operating model

The service should work in six main steps. Each step stays simple and has a clear responsibility.

1. Intake Request enters the company app.	**2. Retrieve** Relevant documents and tenant-safe context are gathered.	**3. Plan** Ministral decides which profile and workflow to use.	**4. Execute** Devstral performs engineering work through approved tools.
5. Validate Validators and tests check the result before it is accepted.	**6. Explain** The service returns a clear business and technical explanation.	**7. Record** Runs, steps, artifacts, and approvals are logged.	

3. Why the architecture uses two models

This service separates decision-making from engineering execution. The orchestrator model handles routing, summaries, and business-facing reasoning, while the worker model handles codelevel and test-level work. This split reduces cost, improves consistency, and makes the service easier to govern.

Layer	Primary model	Main job	Why it matters
Orchestration	Ministral 3 8B	Route tasks, retrieve context, explain results	Keeps workflows structured and business-readable
Engineering	Devstral Small 2 24B	Explore code, edit files, fix tests, produce patches	Provides deeper software-engineering capability
Validation	Backend rules + test tools	Check path scope, test results, approvals	Prevents unsafe outputs from becoming actions

4. Model details: Ministral 3 8B

Ministral 3 8B is the recommended orchestration model for this service. Official documentation describes it as a powerful and efficient model for edge deployment with a 256k context window, native function calling, structured outputs, and strong system-prompt adherence.[1][2]

Model fact	Value
Official role in this architecture	Router, orchestrator, business analyst, readheavy assistant
Published context window	256k
Vendor-highlighted features	Function calling, structured outputs, local/edge deployment, strong system-prompt adherence
Service use	Request routing, profile selection, business explanation, test planning, final summaries
Recommended service operating pattern	Low temperature, smaller default context, retrieval-first workflow

Main responsibilities

- Read the incoming business or product request.
  Decide which agent profile should handle the work.
- Trigger retrieval of the right context pack before execution.
- Produce a short, structured plan for the next stage.
- Summarize outcomes for business and management users.

How it should be used

- **Input:** user request, tenant-safe retrieved context, profile descriptions, and system rules.

- **Output:** a routing decision, a small plan, and a business-readable explanation.
- **Boundary:** it should not be the main model that edits multiple files or performs deep code repair unless acting as a fallback.

Service recommendation

For company operations, Ministral should run with conservative settings and be used as the coordination layer. The architecture should keep its normal runtime context smaller than the maximum window and only expand context for larger document-heavy tasks.

5. Model details: Devstral Small 2 24B

Devstral Small 2 is the recommended engineering worker model. Mistral describes it as an opensource model that excels at using tools to explore codebases, edit multiple files, and power software-engineering agents. Its official model materials describe a 256k context window, Apache 2.0 licensing, and published SWE-bench Verified performance of 68.0% for the 24B variant.[3][4]

Model fact	Value
Official role in this architecture	Engineering worker, coding agent, test-fix worker
Published context window	256k
Vendor-highlighted features	Agentic coding, multi-file editing, tool use for codebases, local deployment suitability
Published benchmark note	68.0% SWE-bench Verified for Devstral Small 2 24B in the official model card
Service use	Frontend/backend implementation, patch generation, test repair, engineering tool loops

Main responsibilities

- Read only the code files and engineering context selected for the task.
- Generate scoped patches across frontend and backend files.
- Interpret failing test output and produce repair attempts.
- Work through approved engineering tools instead of unrestricted shell access.

Return a clear patch summary and technical reasoning for review.

How it should be used

- **Input:** scoped repo context, retrieved requirements, test output, validator rules, and tool responses.
- **Output:** code patches, test updates, structured engineering summaries, and follow-up actions.
- **Boundary:** it should not decide company policy, business workflow routing, or approval outcomes on its own.

Service recommendation

For company operations, Devstral should be used as the main engineering execution model with tighter concurrency controls than the orchestrator model. Large-context runs should be reserved for harder engineering tasks instead of being used by default.

6. Where RAG fits

RAG should sit next to the orchestrator, not inside the frontend and not as a permanent substitute for direct code tools. The orchestrator should decide when retrieval is needed, fetch only the right tenant-safe material, and pass a smaller grounded context packet to the worker model.[5]

- **Good RAG sources:** user stories, process documents, acceptance criteria, API docs, architecture notes, test history, and agent skill documents.
- **Not the main use for RAG:** source code editing. Code should still be handled mainly through repo search/read tools.
- **Tenant rule:** retrieval indexes should use tenant-safe namespaces or equivalent isolation boundaries.

7. Main implementation idea for the company

The service should be delivered as a local AI layer behind the application backend, not as a direct model connection from the user interface.

- **Step 1:** the app sends a request to the backend AI service.
- **Step 2:** the orchestrator retrieves the right context and chooses the right profile.
- **Step 3:** Ministral produces the workflow plan and prepares the engineering handoff.
- **Step 4:** Devstral executes the engineering or testing task through controlled tools.
- **Step 5:** validators and tests confirm the output before the system finalizes the run.
- **Step 6:** the service records the run and returns a clear explanation and result.

8. Why this plan is appropriate for company rollout

- It keeps model hosting local and reduces cloud dependency.

- It limits risk by separating planning from engineering execution.
  It supports about 30 agent profiles without deploying 30 separate models.
- It is easier to govern because retrieval, tools, validation, and approvals stay in backend code.
- It leaves room to harden serving later without changing the product-facing architecture.